

Dual-core machine learning accelerator for attention mechanism

Shreeya Ajay Bhonsle (A69040642) , Chi-Han Chiu (A69036037) , Nikhita Neelakanta (A69041742) ,
Ming-Yang Wu (A16504833) , Bing-Cheng Chiang (A69040193) , Wen-Chieh Lo (A69042225)

Abstract—This report presents the design and implementation of a dual-core machine learning accelerator tailored for attention mechanisms, a core component of modern transformer models. The architecture features an asynchronous clocking scheme with a dual-core setup to parallelize the matrix multiplication and normalization phases. Furthermore, a hardware-optimized sparsity-aware Softmax (SFP) engine is introduced, utilizing row-energy accumulation and cross-core sum exchange to execute accurate fixed-point normalization. We also outline a comprehensive step-by-step verification methodology that spans from single-core RTL design to full-chip hierarchical physical design (PnR) using a 65nm process. Architectural optimizations, such as repipelining the divider and adder tree, dramatically reduced the critical path delay from over 24 ns to under 1 ns, achieving zero timing violations. Finally, our design achieves fully functional gate-level simulations and successful tape-out-ready physical layouts.

I. INTRODUCTION

Attention mechanisms have revolutionized natural language processing and computer vision by enabling models to dynamically weigh the importance of different input elements. At the core of the attention mechanism is the computation of attention scores, which determines how much focus a query should place on various keys. This is mathematically expressed as multiplying a Query matrix (Q) with a transposed Key matrix (K) to obtain raw attention scores. These raw scores are then normalized using a Softmax function, converting them into a valid probability distribution where all weights sum to one. Finally, the normalized attention weights are multiplied by a Value matrix (V) to produce the final output context vectors.

However, the associated matrix multiplication (MatMul) and Softmax normalization operations pose significant computational and memory bandwidth challenges. The Softmax function, in particular, requires global awareness of the entire row (or sequence) to compute the normalizing sum, creating a bottleneck in deeply pipelined hardware. To address these bottlenecks, this project proposes a dual-core hardware accelerator designed specifically for the attention mechanism.

The proposed design leverages two independent cores communicating via asynchronous FIFOs, allowing flexible scheduling and increased throughput. To efficiently handle the Softmax operation, we developed a Sparsity-Aware Softmax (SFP) engine that skips redundant computations for zero or near-zero data, thereby saving power and cycles.

Due to the immense complexity of the dual-core asynchronous design, a rigorous, step-by-step implementation and verification strategy was adopted. This incremental approach, ranging from basic single-core synthesis to full-chip hierarchical place-and-route (PnR), ensured functional correctness at

every stage. This report details both the theoretical architecture of the accelerator and the practical milestones achieved during its physical implementation.

II. PROPOSED ARCHITECTURE

A. Dual-Core System & Asynchronous Clocking

To maximize throughput, the accelerator is partitioned into two distinct processing cores (Core 0 and Core 1). Each core operates in its own clock domain (`clk0` and `clk1`) to simulate realistic system-on-chip (SoC) environments where different functional blocks might run at different frequencies. Data and control signals are securely transferred between the two clock domains using asynchronous FIFOs (Async FIFO). This cross-domain communication (CDC) ensures that matrix chunks processed by one core can be synchronized with the other core without meta-stability issues, particularly during the global sum accumulation required for Softmax normalization.

B. Core Controller & Data Flow

Each core features an independent controller and a memory map (`reg_map`) to handle top-level configurations. The controller manages the data flow between the Query Memory (QMEM), Key Memory (KMEM), the MAC array, and the SFP engine based on the defined operation mode (`op_mode`):

- **Mode 1 (MatMul Only):** The core multiplies data from QMEM and KMEM using the MAC array, and directly stores the result into the Partial Sum Memory (PMEM).
- **Mode 0 (MatMul + Normalization):** Data from QMEM and KMEM are multiplied, and the resulting partial sums are fed into the SFP engine. The normalized outputs are then written back to either PMEM or KMEM, dictated by `sfp_write_to_pmem` and `sfp_write_to_kmem` flags.

The controller automatically generates status flags (e.g., `qmem_free`) to notify upper-level modules when memory banks are available for new data, enabling efficient cycle-saving data prefetching.

C. Sparsity-Aware Softmax (SFP) Engine

The SFP row block is responsible for normalizing one row of the MAC array outputs. The normalization process is divided into two pipelined phases:

- 1) **Accumulation (acc):** When `acc_start` is pulsed, the engine computes the row energy $S_i = \sum_j |sfp_in[j]|$ using an adder tree. The sum is temporarily stored

in an internal FIFO, and `acc_done` is asserted upon completion.

- 2) **Division (div):** During the division phase, triggered by `div_start` (which is gated by a `div_busy` signal to prevent overlapping operations), the engine retrieves the local sum from the FIFO and adds it to the sum from the other core (`sum_in`). It then computes the normalized output:

$$sfp_div_out[c] = \text{trunc} \left(\frac{|sfp_in[c]| \ll out_shift}{S_{local} + sum_in} \right) \quad (1)$$

Upon finishing the division, a `div_done` pulse is generated to indicate that the normalized results are valid.

To further optimize performance, a sparsity gating mechanism is implemented. If the computed row energy S_i is strictly less than a configurable `SFP_THRESHOLD`, the engine bypasses the division and outputs all zeros. This effectively filters out negligible attention scores, conserving dynamic power.

III. DESIGN METHODOLOGY AND VERIFICATION INFRASTRUCTURE

To facilitate rapid development and reliable testing of the dual-core design, a robust Makefile-driven flow and shell-based verification infrastructure were established.

A. Makefile Management

A centralized Makefile provides unified commands for behavioral simulation, synthesis, and gate-level simulation (GLS) across various design targets (e.g., `fullchip`, `core`, `mac`, `sfp_row`). By standardizing the build system, team members could seamlessly run target-specific tests using commands such as `make sim TARGET=core` or `make syn TARGET=mac`, significantly improving development efficiency and avoiding configuration errors.

B. Shell-Based Verification and Automated Parsing

Synthesis quality of results (QoR) and timing closure were continuously tracked using a custom shell-based parser (`parse_reports.sh`). Integrated into the Makefile as the `make parse` target, this tool automatically scans Design Compiler output reports to extract and summarize critical metrics, including total cell area, dynamic power, leakage power, and Worst Negative Slack (WNS). This automated reporting mechanism proved essential during the RTL optimization phase (Step 5), allowing the team to quickly evaluate the impact of repipelining the SFP block on the overall critical path.

IV. STEP-BY-STEP IMPLEMENTATION AND VERIFICATION

To systematically conquer the design complexity, the project was executed in six sequential milestones.

A. Step 1: Single Core RTL, Synthesis & PNR

The baseline single-core design was implemented and verified first. This step established the core infrastructure, including the MAC array and the basic controller. Post-PnR layout, GLS waveform, and terminal output for the single-core design are shown below, with summary metrics in Table I.

To validate the flexibility of the single-core design, we separately tested the QK product and the V-Norm product using the same computation path. This demonstrates that the design can support both stages of the attention computation, providing a verified foundation for later steps where these operations are connected into a more complete pipeline.

In addition, minor modifications were made to the professor-provided testbench to align with our implementation. Specifically, we updated the DUT from `fullchip` to `core`, parameter settings, and data packing format, and added automatic golden result comparison (PASS/FAIL). These changes improve verification efficiency while preserving the intended computation functionality.

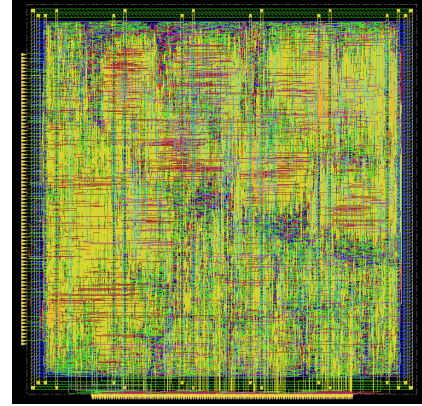


Fig. 1: Single-Core Layout (post-PnR).

```

modelsim> run
##### 0 data txt reading #####
##### 1 data txt reading #####
##### Open writing #####
##### Run writing #####
##### X loading to MAC #####
##### Execute #####
OK phase verification
(row) RTL      col0 col1 col2 col3 col4 col5 col6 col7
[0] RTL      93  -20  -7   94  -48  -45  -63  -86  -96
    golden:
    [OK]
[1] RTL      52  -21  -73   24  -5   -33  -41  -23  -23
    golden:
    [OK]
[2] RTL      38  27  50  63  -28  21  -3  -55  -55
    golden:
    [OK]
[3] RTL      33  -74  17  125  -37  -17  -27  -51  -51
    golden:
    [OK]
[4] RTL      80  -10  -10  13  -138  -37  -90  -56  -56
    golden:
    [OK]
[5] RTL      42  42  1   26  68  7  7  -8  -45  -45
    golden:
    [OK]
[6] RTL      -19  48  99  -7  -1  23  46  -20  -20
    golden:
    [OK]
[7] RTL      -63  41  128  -46  -1  78  74  37  37
    golden:
    [OK]

-----
PASS: all 8 x 8 results correct
Simulation complete via $finish(1) at time 111 NS + 0
/netlist/step1_tb.v:238  #10 $finish:

```

Fig. 2: Single-Core GLS Terminal Output.

B. Step 2: Output Normalization

The single-core MAC array was integrated with the SFP (Softmax with Fixed-Point) row block. The SFP block normal-

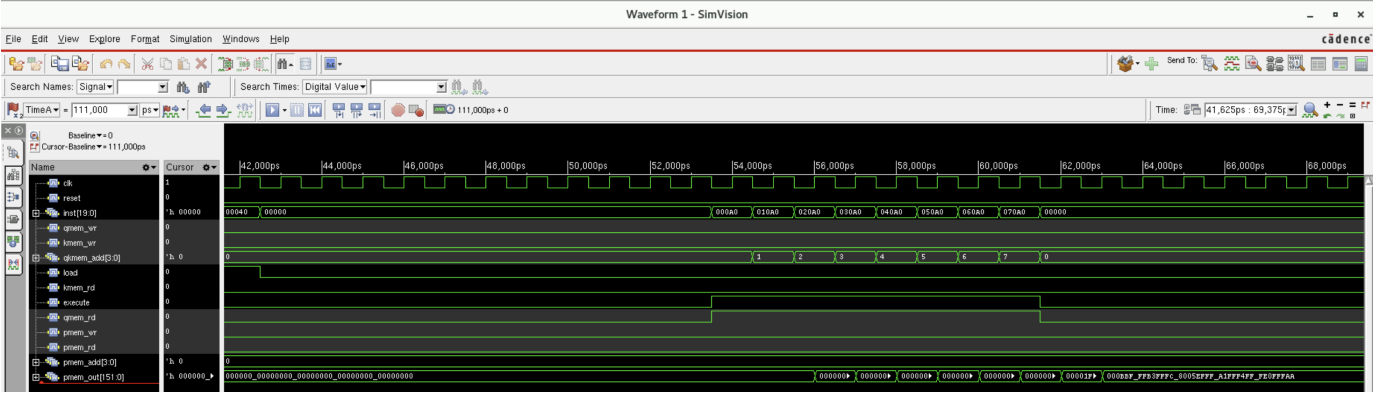


Fig. 3: Single-Core GLS Waveform.

TABLE I: Step 1 Synthesis & PnR Results

Design	Power (VCD)	WNS	DRC
Single Core	50.74 mW	-0.014 ns	0

izes the partial sums row by row in two phases: *accumulation* and *division*.

During the **accumulation (acc)** phase, the block calculates the row energy by summing the absolute values of the incoming partial sums from the MAC array ($S_i = \sum |sfp_in[j]|$).

In the **division (div)** phase, each absolute partial sum is divided by the global row sum to produce the normalized output. To support multi-core scaling, the global sum combines the local core's sum (S_i) with the sum from the other core (sum_in).

Since the theoretical output of the normalization falls within the range $[0, 1]$, integer division would yield zero for almost all values. To address this, a fixed-point scaling mechanism is introduced using the `out_shift` parameter. The numerator is logically left-shifted by `out_shift` bits before division:

$$sfp_div_out[c] = \text{truncate} \left(\frac{|sfp_in[c]| \ll out_shift}{S_i + sum_in} \right)$$

This shift defines the fractional precision of the output. For example, a default `out_shift = 7` allocates 7 bits for the fractional part, mapping the $[0, 1]$ range to $[0, 128]$ in the 8-bit output. Behavioral simulations verified that these calculations correctly normalize the partial sums. Figure 4 shows the simulation waveform demonstrating the functional correctness of the Step 2 design, with the output matching the expected normalized values.

C. Step 3: Hierarchical Synthesis of Core

Memory macros (SRAMs for QMEM, KMEM, and PMEM) were incorporated into the core. A hierarchical synthesis and PnR flow was adopted to manage the physical placement of the SRAM macros alongside standard cells.

D. Step 4: Hierarchical Synthesis of Dual-Core

The design was expanded to a full dual-core architecture. This step focused on non-lockstep execution across the two

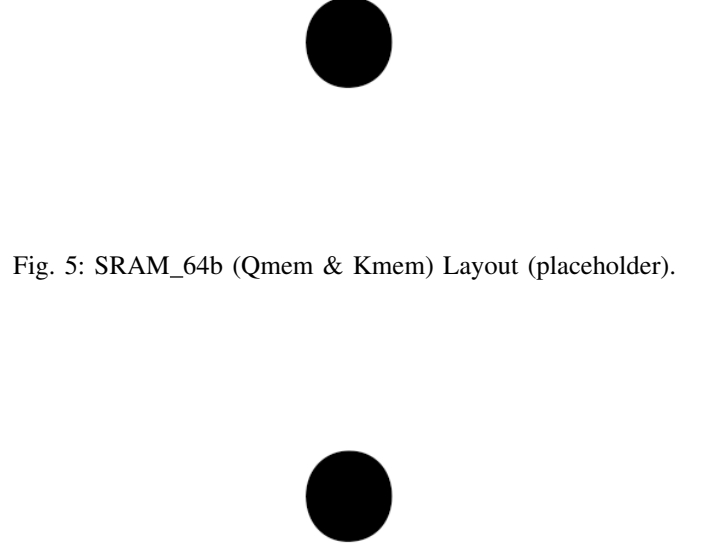


Fig. 5: SRAM_64b (Qmem & Kmem) Layout (placeholder).

Fig. 6: SRAM_160b (Psum_mem) Layout (placeholder).

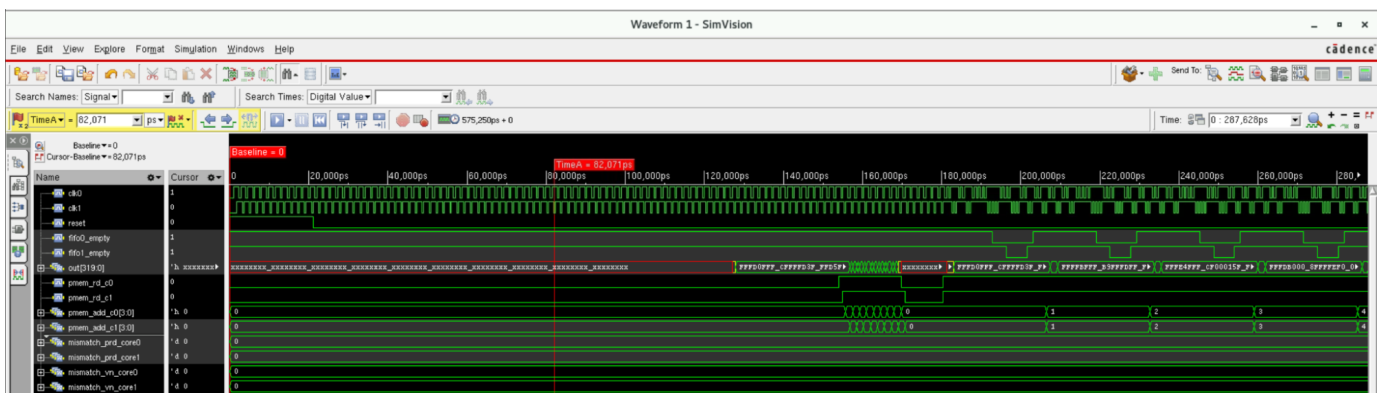
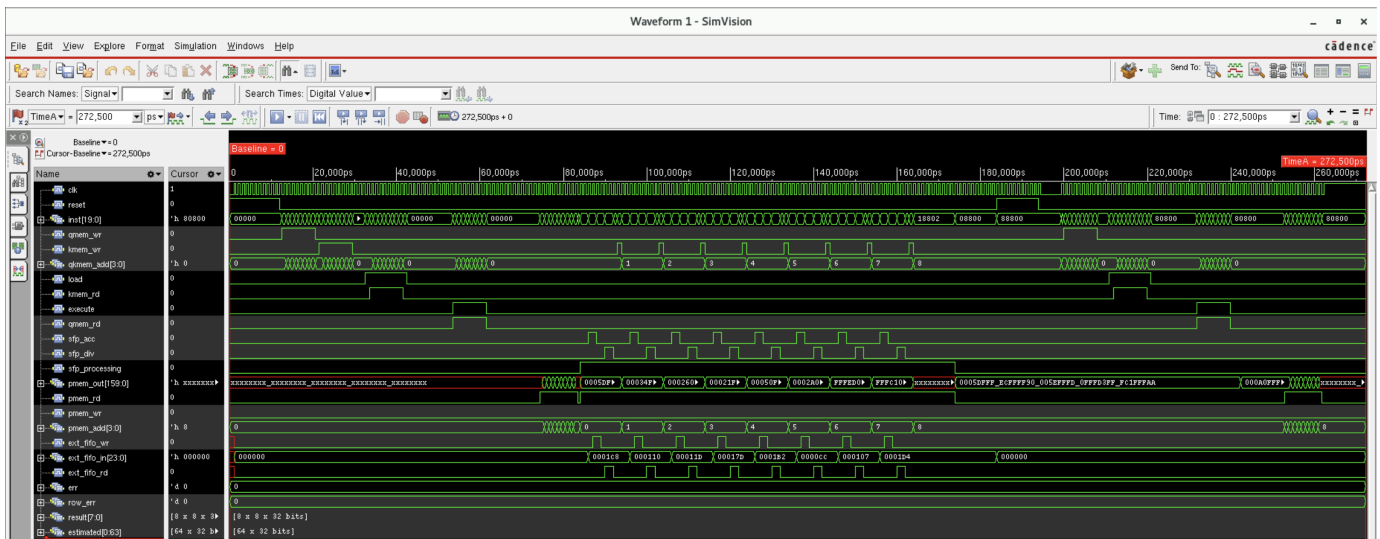
clock domains. Hierarchical PnR constraints were applied to seamlessly integrate both cores at the top level.

E. Step 5: Alphas (Architecture Optimization)

Initial synthesis results highlighted a massive timing violation (Worst Negative Slack ≈ -23.9 ns) primarily due to

TABLE II: Step 3 Hierarchical Core Results

Design	Power (VCD)	WNS	DRC
Hier. Core	61.22 mW	—	—



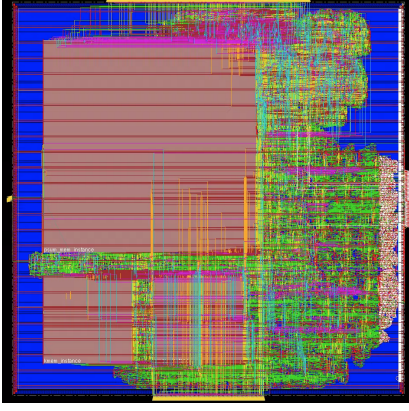


Fig. 7: Hierarchical Core Layout.

```

#### execute ####
VN phase verification start (checking psm content)
#### single psm content & compare to golden ####
[ row] RTL      col0  col1  col2  col3  col4  col5  col6  col7
golden:
[0] RTL      : 160 -103 155 232 -311 283 -242 -79
    golden: 160 -103 155 232 -311 283 -242 -79
    [OK]
[1] RTL      : 238 85 98 127 334 179 54 75
    golden: 238 85 98 127 334 179 54 75
    [OK]
[2] RTL      : -214 -333 -334 46 -430 -300 -281 -253
    golden: -214 -333 -334 46 -430 -300 -281 -253
    [OK]
[3] RTL      : -517 -433 -354 -314 -526 -504 -309 -381
    golden: -517 -433 -354 -314 -526 -504 -309 -381
    [OK]
[4] RTL      : -311 40 -233 -253 -196 -396 298 68
    golden: -311 40 -233 -253 -196 -396 298 68
    [OK]
[5] RTL      : -419 -414 -467 -366 -456 -465 -371 -432
    golden: -419 -414 -467 -366 -456 -465 -371 -432
    [OK]
[6] RTL      : -154 -21 -47 -73 140 -267 187 3
    golden: -154 -21 -47 -73 140 -267 187 3
    [OK]
[7] RTL      : -201 24 -35 -201 -351 42 -58 -38
    golden: -201 24 -35 -201 -351 42 -58 -38
    [OK]
-----
PASS: 8 rows x 8 cols all match estimated result
Simulation complete via $finish(1) at time 272500 PS + 0
/netlist/core_tb.v:603 #10 $finish;

```

Fig. 9: Hierarchical Core GLS Terminal Output.



Fig. 10: Fullchip Layout (placeholder; replace with fullchip snapshot).

```

===== OVERALL SUMMARY =====
Mode: NON-LOCKSTEP, split 40-bit inst bus
clk0=1.000GHz (core0) clk1=1.333GHz (core1)
tick0 then tick1 per step - 1 posedge per core per operation
OK phase -- CORE0: 0 CORE1: 0 mismatch(es)
VN phase -- CORE0: 0 CORE1: 0 mismatch(es)
>>> ALL TESTS PASSED <<<
=====
Simulation complete via $finish(1) at time 575250 PS + 0
/netlist/fullchip_tb.v:714 #10 $finish;\r
xcelium>

```

Fig. 12: Dual-Core GLS Terminal Output.

TABLE III: Step 4 Dual-Core Results

Design	Power (VCD)	WNS	DRC
Dual Core	19.19 mW	—	—

deep combinational logic in the SFP row's absolute sum and division calculations. To achieve timing closure for a 1.0 ns clock period, we added 3-stage pipeline in the accumulation and also add `-retime` to meet the timing requirement:

- **Divider Optimization:** Previous synthesis timing reports indicate that the native Verilog / divider is the bottleneck of achieving 0 WNS. Therefore, we implemented 2 possible upgrades of our division module.

- 1) 6-stage pipelined long divider
- 2) 10-cycle MCP native Verilog / divider

Besides, the 2 designs share a common handshake interface with the core (div_start, div_busy, div_done); macros (SFP_LONGDIV, SFP_MCP, VANILLA) are also defined to compile and synthesis the designs with ease.

Results. Both designs passed the Hold/Set up time check and the gate-level simulation. Nonetheless, we have to select the optimal design for PnR. Hence, we compare the synthesis result in Figure 13. The 6-staged pipelined long division hardware outperforms the 10 cycle MCP Verilog / divider with (~23%) lower area, slightly lower power, and much shorter synthesis time (10 min vs. 45 min). Thus, we select the pipelined long divider for PnR.

Setup	Corner = TC Target Freq = 1G Fullchip.v	Pipelined Long Div	MCP / Operator
RTL Design	Latency	6cyc	10cyc
Timing	BC Hold WNS	0.068	0.060
	WC Setup WNS	0.000	0.000
	TC Setup WNS	0.000	0.000
Area(um^2)	Total	291242.5	376123.7
	Combinational	133760.2	228015.0
	Noncombinational	157482.4	148108.7
	Buf/Inv	4601.2	15221.2
Power(mW)	Total	65.2	67.5
	Internal	57.6	59.6
	Switching	6.1	6.1
	Leakage	1.5	1.8
Observation	Syn Time(min)	10	45
Validation	GLS	Pass	Pass
Decision		Go for PnR!	

Fig. 13: Synthesis result of core with different division module

- **Pipelining: Adder Tree Optimization:** The row energy accumulator was redesigned as a 3-stage pipelined reduction tree. After the 8 parallel multipliers, one pipeline stage captures the multiplication results, the next stage performs four pairwise additions, and the final stage reduces the partial sums to produce the accumulated row energy.
- **Chip Interface Design and Internal Control:** The purpose of redesigning the chip interface is not only to remove redundant signals, but also to hide unnecessary internal details from the user. The interface should be

intuitive and self-consistent, so that users can operate the chip without understanding internal dataflow or timing dependencies. Each core is independently controlled. We describe the interface usage from the perspective of a single core.

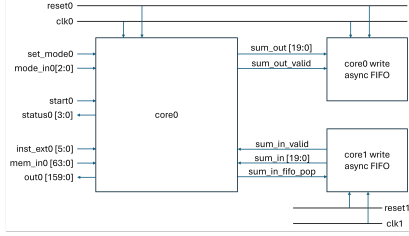


Fig. 14: Core interface redesign

Example Usage: MULT + NORM + READ.

A user wants to compute matrix multiplication, apply normalization, save the result in KMEM for later use, but also want to read it out for verification.

- 1) Write data to KMEM and QMEM through `mem_in` by designating `inst_ext[5:0]` (SRAM operation & addr).
- 2) Set `mode_in = 3'b001` (MULT+NORM → KMEM and PMEM) and assert `set_mode` for one cycle to latch the configuration.
- 3) Assert `start` for one cycle to begin execution. If the core is busy (`status[3]=1`), the `start` signal is ignored.
- 4) During execution, `status[3]=1` indicates the core is busy, while `status[2:0]` indicate memory locks (QMEM / KMEM / PMEM).
- 5) User may start to write K/QMEM data for next operation after the K/QMEM lock is freed.
- 6) `status[3]` returns to 0 after the result is stored in PMEM and KMEM. The user can then issue PMEM read by designating `inst_ext[5:0]` (SRAM operation & addr) and wait for 1 cycle for the result at the `out` port.

The above example involves external memory instructions and core mode setting. Refer to Figure 15 for details.

Signal Name	Dir	Width	Category	Description	Encoding / Behavior
set_mode	In	1	CTRL	Mode latch trigger: 1-cycle pulse. Latch mode_nron rising edge	000: MULT+PMEM
mode_in[2:0]	In	3	CTRL	Core operation mode: 000: MULT+NORM+PMEM & result destination 001: MULT+NORM+KMEM 010: MULT+NORM+PMEM+KMEM 011: MULT+NORM+PMEM+KMEM others: Reserved, will not take effect	000: MULT+NORM+PMEM 001: MULT+NORM+KMEM 010: MULT+NORM+PMEM+KMEM 011: MULT+NORM+PMEM+KMEM
start	In	1	CTRL	Start execution: 1-cycle pulse, ignored if Core busy=1	00: NO_OP 01: KMEM_WRITE 10: QMEM_WRITE 11: PMEM_READ
status[3]	Out	1	STATUS	Core busy: 1: executing, 0: idle	
status[2]	Out	1	STATUS	QMEM locked: 1: QMEM in use, ignore Ext mem command QMEM_WRITE	
status[1]	Out	1	STATUS	KMEM locked: 1: KMEM in use, ignore Ext mem command KMEM_WRITE	
status[0]	Out	1	STATUS	PMEM locked: 1: PMEM in use, ignore Ext mem command PMEM_WRITE	
inst_ext[5:0]	In	2	MEM	Ext mem command	00: NO_OP 01: KMEM_WRITE 10: QMEM_WRITE 11: PMEM_READ
inst_ext[5:2]	In	4	MEM	Memory address: Address for external access	
mem_in[63:0]	In	64	MEM	Write data input: Used for KMEM/QMEM write	
out[159:0]	Out	160	MEM	Read data output: Output of PMEM read	
(Notes)					
MULT: Result = QMEM * Transpose(KMEM)					
MULT+NORM: Result = Norm(QMEM * Transpose(KMEM))					

Fig. 15: Available core instruction/operation specification

Design Insights.

- 1) **Flexible data loading, fixed execution.** The user has full freedom to write Q/K memory in any order and timing. In contrast, execution follows a fixed

address increment pattern, since the hardware performs a deterministic MAC sequence. This cleanly separates user control from hardware scheduling.

- 2) **Deterministic memory locking.** After start, memories are locked and released in a fixed order. Once a memory is released, it will not be locked again before the core becomes idle. This gives a clear contract: unlocked memory is always safe to write.
- 3) **Hardware-enforced protection.** Although status signals are visible, correctness does not rely on the user. Invalid operations (e.g., writing to locked memory or starting while busy) are automatically ignored by hardware.
- 4) **Unified memory interface with minimal command set.** All external memory accesses share the same interface (`addr`, `data_in`, `data_out`). To avoid conflicts, only four commands are allowed: NO_OP, KMEM_WRITE, QMEM_WRITE, PMEM_READ. This eliminates read/write conflicts and simplifies controller design.
- 5) **Mode-driven dataflow.** A 3-bit mode encodes both operation type and result destination. Results can be written to PMEM for output, or kept in KMEM for further computation. This enables chaining operations without extra data movement.
- 6) **Implicit pipeline support.** The interface naturally allows overlapping computation and data loading. While some memories are locked for computation, others can already be reused for the next stage. For example, in MULT+NORM mode, QMEM is released after MAC load, so if user's next step is multiply norm by V, user can start writing V data while normalization is still running.

- **Production System Setup (Makefile-based Flow):** A centralized Makefile is designed to manage the entire simulation pipeline, including RTL simulation, gate-level simulation (GLS), and synthesis. Key parameters such as simulation target, clock period, and design configuration can be specified through command-line arguments, enabling reproducible experiments and fast design iteration. A summary of the supported Makefile commands and arguments is provided in Table IV.

Particularly, the `USER_DEFINES` variable (which can be stored persistently in `USER_DEFINE_TASK_VARS`) offers a significant advantage for architectural exploration. It allows developers to seamlessly switch hardware configurations such as changing the division module to `SFP_LONGDIV` or `SFP_MCP`, or enabling sparsity features via `SFP_THRESHOLD` without modifying the underlying RTL or synthesis scripts. Because this single variable propagates globally to both the simulator (iverilog) and the synthesizer (Design Compiler), it guarantees functional consistency between RTL verification and the final synthesized netlist.

- **Randomized Input Generation:** We support automatic generation of randomized test patterns for functional verification. The system detects missing pattern files and

TABLE IV: Supported Makefile Commands and Parameters

Command / Parameter	Description
make sim	Runs RTL simulation using iverilog and vvp. Supports SV via -g2012.
make gls	Runs gate-level simulation using post-synthesis netlists (syn/gate/*.out.v) and PDK.
make syn	Runs Design Compiler synthesis using run_dc.tcl.
make sim-shell	Launches the interactive '.mingu' shell for testbench execution.
make parse	Parses synthesis reports and outputs a summary of area, power, and timing.
TARGET=<name>	Specifies the design target (e.g., fullchip, core, mac, sfp_row).
SYN_EFFORT=<level>	Sets synthesis optimization effort: low, medium, or high (default).
CYCLE=<val>	Overrides the default clock period during simulation.
USER_DEFINES=<...>	Passes custom Verilog macros (e.g., SFP_LONGDIV) to both simulation and synthesis.

generates them on demand, allowing large-scale testing without manual data preparation. This improves coverage and helps uncover corner-case bugs under diverse input conditions.

- **.mingu Shell (Matrix INstruction Generation Utility):**

The crown jewel of our verification framework is a custom, highly extensible simulation shell named the *Matrix INstruction Generation Utility* (.mingu). Built on top of our Makefile flow and utilizing the hardware’s unified interface, it completely abstracts away low-level testbench manipulation, allowing users to express verification tasks in a clean, scriptable format.

The .mingu shell operates in a two-phase execution model:

- 1) **Configuration Phase (Before `fix_set`):**

Users define the environment using shell-level commands. This includes setting the simulation target (`set_scope core|fullchip`), configuring the clock periods for asynchronous testing (`set_clock_period`), selecting the backend execution stage (`set_sim_stage sim|gls|post_sim`), defining user variables (`set VAR value`), and controlling output features (`set_dump_vcd`).

- 2) **Execution Phase (After `fix_set`):** Once the environment is locked and compiled via `fix_set`, the shell enters the interactive or batch testbench phase. Users issue high-level operations that perfectly map to our hardware’s clean controller interface. Support includes data loading (`writeQ`, `writeK0`, `writeK1`), mode configuration (`set_exec_target`), operation triggers (`simulate`, `reset`), and automated golden model verification (`verifypmem`).

Crucially, the shell supports **variable substitution** and **for-loops**, which makes large-scale automated testing remarkably simple. As shown in the comprehensive list in Table V, we utilize this feature extensively across both the Alpha and Step6 workspaces to ensure complete

functional coverage.

For example, Figure 16 demonstrates the generation of 100 randomized data patterns and output verification via the .mingu shell. Because the hardware interface is well-defined and minimal, these high-level commands can directly correspond to meaningful hardware states. This provides a highly scalable and resilient testing framework that easily transitioned our tests from basic single-core RTL all the way to full-chip, dual-clock Post-PnR netlists without writing new testbenches.

```
# In-n-out100-fullchip.mingu - Batch run random100 pattern sets (0..99) [fullchip_shell]
#
# Dual-core: writeQ/writeK1. Uses same data for both cores (or gen_random_patterns --dual_core).
#
# USAGE: ./sim_shell.sh -f in-n-out100-fullchip.mingu
# (auto-runs make gen_patterns if sw/pattern/random100/ is missing)
#
set PATTERN sw/pattern/random100
set output_dir simwaveform
set_sim_stage sim
set_clock_period 1
set_sim_defines SFP_LONGDIV
set_sim_target fullchip_shell
set_dump_vcd 0
fix_set

for i = 0 to 99
  reset

  # Test 1: Q*KT
  set_exec_target CORE_MAC1_mac_to_PREN
  writeQ $(PATTERN)/data_$i.txt
  writeK0 $(PATTERN)/data_$i.txt
  writeK1 $(PATTERN)/data_$i.txt
  simulate
  verifypmem

  # Test 2: Norm(Q*KT)
  set_exec_target CORE_MAC1_NORM_mac_to_PREN_and_PREN
  writeQ $(PATTERN)/data_$i.txt
  writeK0 $(PATTERN)/data_$i.txt
  writeK1 $(PATTERN)/data_$i.txt
  simulate
  verifypmem

  # Test 3: V*Norm(T)
  set_exec_target CORE_MAC1_mac_to_PREN
  writeQ $(PATTERN)/data_$i.txt
  writeK0 $(PATTERN)/data_$i.txt
  writeK1 $(PATTERN)/data_$i.txt
  simulate
  verifypmem
done
```

Fig. 16: in-n-out100-fullchip.mingu execution script demonstrating for-loops and automated testing.

- **Dual Core Verification on both Lockstep & Non-Lockstep Clock Pair:**

These architectural optimizations successfully reduced the data arrival time from ~25.0 ns down to 0.97 ns, eliminating all timing violations. As shown in Table VI, the fullchip area was reduced by 47.8%, the MAC array by 40.5%, and the SFP row by 54.1% as the costly combinational divider was replaced with a pipelined design.

TABLE VI: Timing Optimization Synthesis Results (Target Period = 1.0 ns)

Design	Before Repipelining		After Repipelining		Area Reduct
	Area (μm^2)	WNS (ns)	Area (μm^2)	WNS (ns)	
Fullchip	557,909	-9.11	291,243	+0.04	-47.8%
MAC Array	116,808	-1.61	69,490	+0.00	-40.5%
SFP Row	53,864	-23.92	24,698	+0.00	-54.1%

F. Step 6: Sparsity-Aware Attention

The final milestone validated the sparsity-aware feature. By injecting highly sparse matrices, we confirmed that the SFP engine correctly identifies rows below `SFP_THRESHOLD` and outputs zeros, bypassing unnecessary long division cycles.

Sparsity Definition and Threshold Calibration. We define the row-wise accumulation as:

$$S_i = \sum_j |(QK^T)_{i,j}|. \quad (2)$$

TABLE V: Summary of standard .mingu scripts and their purposes

Script Name	Purpose & Features
in-n-out-fullchip.mingu	Basic RTL simulation pipeline executing $Q \cdot K^T$, Norm, and $V \cdot \text{Norm}^T$ sequentially on a single static test pattern.
in-n-out100-fullchip.mingu	Large-scale automated RTL simulation utilizing the <code>for</code> loop. It automatically triggers the python/shell pattern generator to create 100 sets of randomized Q, K, and V matrices and verifies the hardware against a software golden model continuously.
in-n-out500-fullchip.mingu	Stress-testing variant scaling the loop and automatic pattern generation to 500 unique datasets.
in-n-out*-fullchip-gls.mingu	Post-PnR validation scripts. By simply changing <code>set_sim_stage</code> to <code>post_sim</code> , the exact same test sequence is piped into <code>xrun</code> with the synthesized and routed netlist (with back-annotated delays or threshold logic).

A row is skipped if $S_i < T$, where T corresponds to `SFP_THRESHOLD`. To achieve a target sparsity of 30%, T is obtained from the empirical distribution of S_i using Monte Carlo simulation over 10,000 datasets:

$$T = \text{Percentile}_{30}(S), \quad (3)$$

ensuring approximately 30% of rows satisfy $S_i < T$. The sparsity is defined as:

$$\text{sparsity} = \frac{1}{N} \sum_i \mathbf{1}(S_i < T). \quad (4)$$

RTL Skip Mechanism and Verification. In RTL, the accumulation from both cores is combined as `sum_2core`, and the skip condition is evaluated via `sum_2core < SFP_THRESHOLD`. When triggered, the divisor is gated to zero (`sum_2core_gated`) and fed into the divider (e.g., `div_longdiv`), allowing its FSM to bypass computation and force zero outputs without modifying the control flow. The same criterion is replicated in the testbench to count skipped rows and compute sparsity.

Results. Using 10,000 samples for searching threshold, the threshold for sparsity we get is 169503. With this threshold, we test test with 1000 other random generated data ,and the measured sparsity we get is 34.975%, close to the 30% target. The deviation is attributed to distribution variance, demonstrating that the threshold generalizes well and that the RTL correctly implements sparsity-aware skipping.

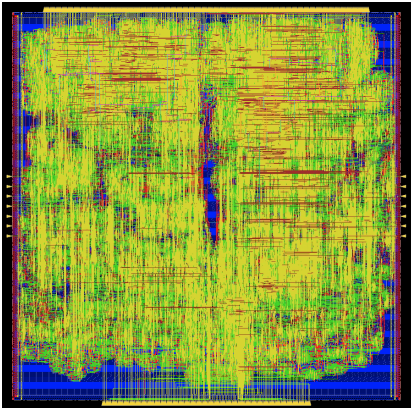


Fig. 17: Sparsity-Aware Attention Dual-Core Layout.



Fig. 19: Sparsity-Aware Attention Dual-Core GLS Terminal Output (placeholder).

V. CONCLUSION

This project successfully demonstrates the design, implementation, and physical realization of a dual-core attention accelerator. By partitioning the system into asynchronous domains, we enabled flexible inter-core data exchange. Through the introduction of a Sparsity-Aware SFP engine, computations on insignificant data are bypassed, promoting energy efficiency. Most importantly, critical timing bottlenecks were identified and resolved via rigorous RTL repipelining, ensuring the design easily meets the 1.0ns clock constraint in a TSMC 65nm process. The progressive, step-by-step verification methodology guaranteed robustness from the behavioral level down to post-PnR gate-level simulations, confirming the design's readiness for physical fabrication.

REFERENCES

- [1] E. Course, "Ece260b final project (placeholder)," Course project report, 2026, replace with real references; use cite in main.tex to list them.

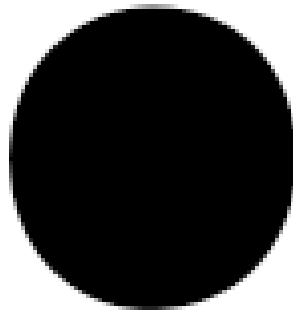


Fig. 18: Sparsity-Aware Attention Dual-Core GLS Waveform (placeholder).

APPENDIX A

MINGU SHELL COMMAND REFERENCE

The `.mingu` shell abstracts the complexity of cycle-accurate testbenches into a high-level scripting language. Below is the complete reference of supported commands, categorized by execution phase.

A. Phase 1: Shell Configuration Commands

These commands must be executed **before** `fix_set`. They configure the underlying simulation backend, Makefile parameters, and output preferences.

- `set_scope <core|fullchip>`: Selects the hardware hierarchy to simulate. Determines whether to instantiate a single core or the dual-core fullchip.
- `set_sim_stage <sim|gls|post_sim>`: Sets the simulation backend.
 - `sim`: RTL behavioral simulation.
 - `gls`: Gate-level simulation using synthesized netlist.
 - `post_sim`: Post-PnR gate-level simulation with delay annotations via `xrun`.
- `set_clock_period <ns0> [ns1]`: Configures the half-cycle clock periods. If `ns1` is omitted, it defaults to `ns0` (lockstep). Example: `set_clock_period 1 1.2`.

- `set_dump_vcd <0|1>`: Toggles waveform dumping (1 for ON, 0 for OFF). Disabling VCD significantly speeds up massive test runs.
- `set VAR value`: Defines a shell variable that can be substituted later using `$(VAR)`. Example: `set PATTERN sw/pattern/random100`.
- `fix_set`: **Required**. Locks all configurations, triggers compilation of the hardware design, and transitions the shell into the execution phase.

B. Phase 2: Hardware Execution Commands

These commands are executed **after** `fix_set`. They map directly to the unified controller interface of the chip.

- `set_exec_target <MODE>`: Sets the execution mode of the hardware controller (mapped to `mode_in`).
 - `CORE_MODE_MULT_save_to_PMEM`: Standard $Q \times K^T$.
 - `CORE_MODE_MULT_NORM_save_to_PMEM_and_KMEM`: Computes $Q \times K^T$, applies Softmax, and saves to both Partial-Sum memory and Key memory for subsequent computation.
- **Memory Writers**: Loads text-based integer matrices into the internal SRAMs via the `inst_ext` and `mem_in` ports.
 - `writeQ <filepath>`: Loads data into QMEM.
 - `writeK0 <filepath>`: Loads data into Core 0's KMEM.
 - `writeK1 <filepath>`: Loads data into Core 1's KMEM.
- `simulate`: Asserts the start signal and waits until the core's `status[3]` (busy flag) goes low. Internally, the testbench computes the golden model equivalent for validation.
- **Golden Checkers**: Reads out the data from the PMEM via the `inst_ext` port and compares it against the golden model.
 - `verifypmem`: Checks PMEM results for all active cores and prints PASS/FAIL statistics.
 - `verifypmem0 / verifypmem1`: Specific checks for Core 0 or Core 1.
- `reset`: Asserts the hardware reset signal for two cycles, clearing internal states.
- `for <var> = <start> to <end> ... endfor`: Iterates block of execution commands. Variables inside the loop can be referenced using `$(var)`. Nested loops are not supported.
- `exit`: Terminates the simulation gracefully and prints a summary of mismatches.